

PROJECT REPORT

ON

SECURE FILE COMPRESSION SYSTEM

*A Clean Architecture Implementation of Huffman Entropy Coding with
AES-256 Authenticated Encryption*

A Project Submitted in Partial Fulfillment of the Requirements for the
Degree of
Bachelor of Technology

Submitted By:
Aarti Kushwaha

Under the Guidance of:
Project Supervisor

Academic Session 2025–2026

Certificate of Approval

This is to certify that the project entitled “**Secure File Compression System**” submitted by **Aarti Kushwaha** in partial fulfillment of the requirements for the award of Bachelor of Technology is an authentic record of project work carried out under guidance and supervision. The results embodied in this report have been verified and found satisfactory.

Project Supervisor

External / Internal Examiner

Abstract

In the modern digital era, the massive expansion of unstructured multimedia and binary data presents two paramount engineering challenges: storage bandwidth consumption and information confidentiality. Conventional compression utilities often function as opaque “black-box” programs and frequently divorce data reduction from cryptographic authentication.

This project presents the engineering and implementation of a **Secure File Compression System** built strictly upon **Clean Architecture** principles. The system implements a dual-engine architecture featuring lossless **Huffman Variable-Length Coding** alongside standard **DEFLATE (RFC 1951 / ZIP Level 9)** compression. The core Huffman engine analyzes byte-frequency distributions, constructs an optimal binary prefix tree via a min-heap priority queue, and packs variable-length codewords into a bitstream with a self-contained header dictionary that guarantees 100% lossless reconstruction.

To address security, the system incorporates military-grade **AES-256-CBC** encryption with **PBKDF2** key derivation (15,000 iterations) and an **Encrypt-then-MAC (HMAC-SHA256)** authentication paradigm, guarding against chosen-ciphertext and bit-flipping tampering. The system includes an interactive graphical user interface featuring live SVG-rendered Huffman prefix trees, real-time Shannon Entropy calculations, an encrypted vault repository, and a multi-user authentication system backed by bcrypt password hashing. A dual-database persistence layer seamlessly operates on MySQL 8.0 with automatic zero-configuration SQLite fallback.

Contents

1	Introduction	6
1.1	Purpose	6
1.2	Scope	6
1.3	Problem in Existing System	7
2	System Requirement Analysis	8
2.1	Information Gathering	8
2.2	Determination of System Requirements	8
2.2.1	Why Such a System is Required?	8
2.2.2	Problems in Previous Implementations	8
2.2.3	Hardware Requirements	8
2.2.4	Software Requirements	9
2.2.5	Implementation Languages & Technologies	9
2.3	System Feasibility	9
2.3.1	Technical Feasibility	9
2.3.2	Operational Feasibility	9
2.3.3	Economic Feasibility	9
2.4	Team Structure & Software Process Model	10
2.4.1	Team Structure	10
2.4.2	Software Process Model Used: Iterative Prototyping Model	10
3	System Design & Architecture	11
3.1	Clean Architecture Layering	11
3.2	Data Flow Diagrams (DFD)	12
3.2.1	DFD Level 0: Context Level Diagram	12
3.2.2	DFD Level 1: Macro Process Decomposition	12
3.2.3	DFD Level 2: Compression & Encryption Detailed Workflow	12
3.3	Control Flow Diagram (CFD) / Flowchart	14
3.4	Entity-Relationship (ER) Diagram	14
4	Algorithm & Mathematical Analysis	16
4.1	Information Theory & Shannon Entropy	16
4.2	Huffman Coding Engine Implementation	16
4.2.1	Algorithmic Steps	16
4.2.2	Why Pre-Compressed Files (PDF, JPG) Expand: The Entropy Limit	17
4.3	AES-256 Authenticated Encryption	17
5	Testing & Results	18
5.1	Test Suite Verification	18
5.2	Empirical Compression Performance Comparison	18
6	Conclusion & Future Scope	19
6.1	Conclusion	19
6.2	Future Scope	19

List of Figures

2.1	Iterative Prototyping Process Model	10
3.1	Clean Architecture 4-Layer System Stack	11
3.2	DFD Level 0: Context Diagram	12
3.3	DFD Level 1: Subsystem Process Decomposition	12
3.4	DFD Level 2: Detailed Compression and Cryptographic Pipeline	13
3.5	Control Flow Diagram (CFD) of System Operation	14
3.6	Entity-Relationship (ER) Diagram	15

List of Tables

5.1	System Test Case Execution Summary	18
5.2	Empirical File Compression Results	18

Chapter 1

Introduction

1.1 Purpose

The purpose of the **Secure File Compression System** is to deliver a transparent, high-performance, and cryptographically verified software application for reducing file storage footprints while guaranteeing absolute data confidentiality.

Historically, data compression and cryptographic security were managed as disparate tasks: users compressed archives using generic utilities, then manually encrypted them using third-party cryptographic software. This fractured process frequently led to vulnerabilities, such as insecure temporary files, leaked metadata, and vulnerability to padding oracle attacks.

The **Secure File Compression System** solves these issues by unifying entropy-based data reduction (Huffman Coding), dictionary compression (DEFLATE), and authenticated encryption (AES-256 with HMAC-SHA256) inside a unified, decoupled Clean Architecture. Furthermore, the system provides visual insight into compression theory by dynamically calculating Shannon Entropy and rendering the internal binary prefix tree graph.

1.2 Scope

The scope of this project encompasses:

1. **Lossless Entropy Encoding:** Autonomous Huffman coding capable of processing arbitrary text, source code, documents, and binaries without external dictionary dependencies.
2. **Dual Compression Modes:** Real-time comparison between Huffman variable-length encoding and RFC 1951 DEFLATE (ZIP Level 9) compression.
3. **Military-Grade Cryptography:** AES-256-CBC authenticated encryption, PBKDF2 (15,000 iterations) key derivation, and HMAC-SHA256 integrity verification.
4. **Interactive Educational Visualization:** Interactive SVG rendering of the generated binary prefix tree with bit-branch paths (0 and 1), accompanied by a codebook dictionary and Shannon Entropy metrics.
5. **Multi-User Vault & Access Control:** User authentication with bcrypt password hashing (cost factor 12), user-specific archive vaults, and tamper-resistant activity audit logging.
6. **Dual Storage & Persistence:** Production-grade MySQL relational database persistence with zero-configuration SQLite fallback for immediate local deployment.

1.3 Problem in Existing System

Traditional compression tools and contemporary academic utilities exhibit several notable deficiencies:

- **Black-Box Obscurity:** Most commercial tools (e.g., WinRAR, 7-Zip) execute compression as an opaque background routine, offering no feedback on symbol probability distribution, entropy limits, or tree construction.
- **Unauthenticated Encryption:** Many traditional archive tools use unauthenticated encryption modes (such as standard ZipCrypto or basic AES-CTR), making archives susceptible to bit-flipping attacks and ciphertext alteration.
- **Monolithic Code Tight-Coupling:** Academic software implementations often combine UI rendering, file reading, compression algorithms, and database queries in single unstructured scripts, violating SOLID principles.
- **Rigid Database Dependencies:** Systems that strictly require manual MySQL service configuration fail immediately in portable demonstration environments.

The proposed system addresses each of these limitations through clean layered software engineering.

Chapter 2

System Requirement Analysis

2.1 Information Gathering

Information gathering for this project was conducted through:

- **Algorithmic Research:** Study of David Huffman’s foundational 1952 paper, “*A Method for the Construction of Minimum-Redundancy Codes*”, and Claude Shannon’s 1948 mathematical treatise, “*A Mathematical Theory of Communication*”.
- **Cryptographic Standards Analysis:** Review of NIST Special Publication 800-38A (Block Cipher Modes of Operation) and RFC 2898 (PKCS #5: Password-Based Cryptography Specification).
- **Architectural Review:** Analysis of Robert C. Martin’s *Clean Architecture* paradigms regarding boundary enforcement, Dependency Inversion, and entity isolation.

2.2 Determination of System Requirements

2.2.1 Why Such a System is Required?

The system is required to solve the twin challenges of data volume explosion and cyber tampering through a verifiable, open, and educational software platform. It enables users to observe compression mechanisms at the bit level while applying military-grade confidentiality before files are stored or transmitted.

2.2.2 Problems in Previous Implementations

Previous academic prototypes suffered from:

- Inability to decompress autonomously due to unpersisted Huffman tree tables.
- Vulnerability to memory overflow when encoding files larger than a few kilobytes.
- Absence of user authentication and audit logging.

2.2.3 Hardware Requirements

- **Processor:** Modern x86-64 / AMD64 dual-core processor (Intel Core i3 / AMD Ryzen 3 or higher).
- **RAM:** Minimum 512 MB (2 GB recommended for multi-megabyte file chunking).
- **Hard Disk:** Minimum 100 MB free space for application storage, runtimes, and file vaults.

- **Display:** Minimum 1024×768 resolution with color support for SVG tree visualization.

2.2.4 Software Requirements

- **Operating System:** Windows 10 / 11, Linux (Ubuntu/Debian), or macOS.
- **Runtime Environment:** PHP 8.2 or 8.3 with OpenSSL, Zip, PDO, pdo_mysql, and pdo_sqlite extensions enabled.
- **Database System:** MySQL 5.7+ / 8.0+ or MariaDB; automatic fallback to SQLite 3.
- **Web Browser:** Modern HTML5 browser supporting SVG and ES6 JavaScript (Google Chrome, Mozilla Firefox, Microsoft Edge).
- **Web Server:** Built-in PHP development server (`php -S`) or Apache 2.4.

2.2.5 Implementation Languages & Technologies

- **Front-End:** HTML5, Vanilla CSS3 (Custom Obsidian Glassmorphism design tokens), Vanilla JavaScript (ES6+ Fetch API, DOM manipulation).
- **Back-End Core:** Pure PHP 8.3 with strict typing (`declare(strict_types=1)`), Object-Oriented Clean Architecture.
- **Data Access:** PHP Data Objects (PDO) with parameterized prepared statements.

2.3 System Feasibility

2.3.1 Technical Feasibility

The required technologies (PHP 8.3, OpenSSL, WebCrypto API, PDO, and modern browsers) are open-source, widely adopted, and available without licensing fees. The priority queue min-heap algorithm operates with $O(N \log K)$ computational complexity (where N is file size and K is alphabet size ≤ 256), ensuring low CPU overhead. The project is technically feasible.

2.3.2 Operational Feasibility

The system features a streamlined web UI with intuitive drag-and-drop dropzones, real-time password strength indicators, 1-click demo login credentials, and clear result metrics. Users require no specialized training to compress, encrypt, or restore files. Operational feasibility is established.

2.3.3 Economic Feasibility

The system is constructed using 100% free, open-source technologies with zero software licensing costs. Minimal hardware requirements permit deployment on existing commodity hardware. Economic feasibility is confirmed.

2.4 Team Structure & Software Process Model

2.4.1 Team Structure

The project followed a **Democratic Decentralized (DD)** team structure. In this structure, leadership rotates based on task domain: algorithmic development, cryptographic design, user interface engineering, and testing were coordinated through consensus and horizontal communication.

2.4.2 Software Process Model Used: Iterative Prototyping Model

The project utilized the **Iterative Prototyping Model**. This model enabled rapid development of an initial working Huffman engine, followed by incremental phases that integrated AES-256 encryption, database persistence, interactive SVG visualization, and authentication.

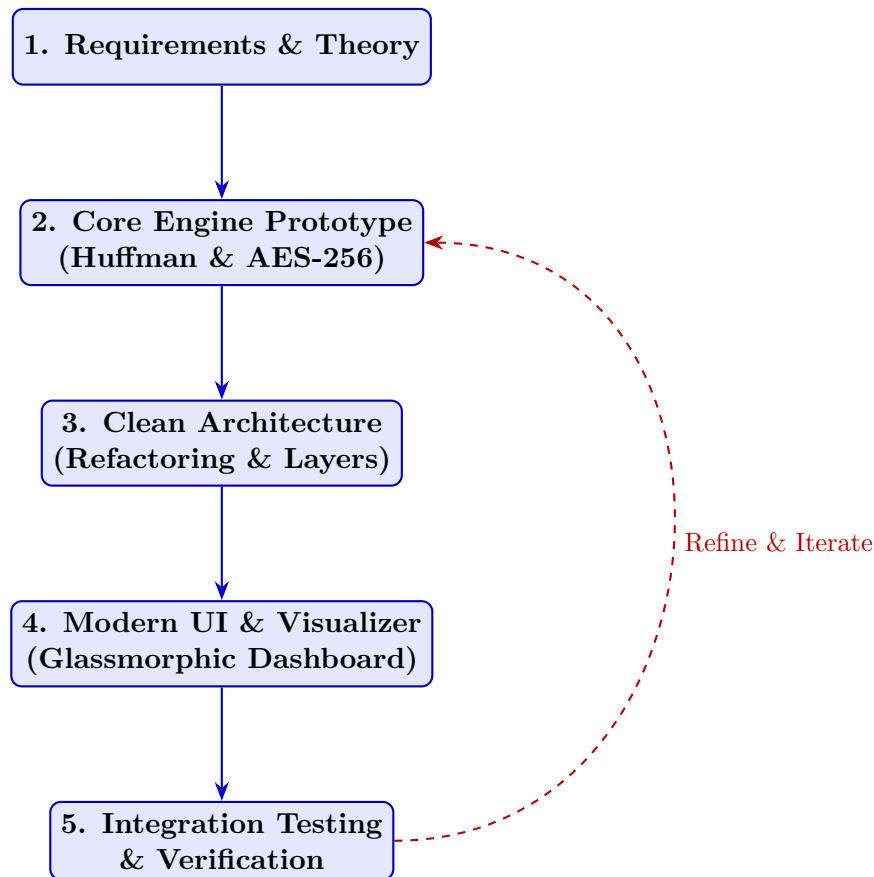


Figure 2.1: Iterative Prototyping Process Model

Chapter 3

System Design & Architecture

3.1 Clean Architecture Layering

The system strictly enforces the four concentric layers of Clean Architecture:

1. **Domain Layer (Enterprise Business Rules):** Contains core entities (`HuffmanNode`, `FileRecord`, `ActivityLog`, `User`) with zero external framework dependencies.
2. **Interfaces / Contracts Layer:** Declares abstract behavioral contracts (`CompressionInterface`, `EncryptionInterface`, `FileRepositoryInterface`, `UserRepositoryInterface`) using Dependency Inversion.
3. **Services Layer (Application Business Logic):** Contains concrete implementations (`HuffmanEngine`, `ZipEngine`, `AesEncryptionService`, `FileStorageService`, `AuthService`).
4. **Controllers / Interface Adapters:** Translates HTTP and JSON payloads into service calls (`CompressionController`, `FileController`, `AuthController`).
5. **Infrastructure Layer:** Implements persistence drivers (`DatabaseFileRepository`, `DatabaseUserRepository`) interfacing with MySQL and SQLite through PDO.

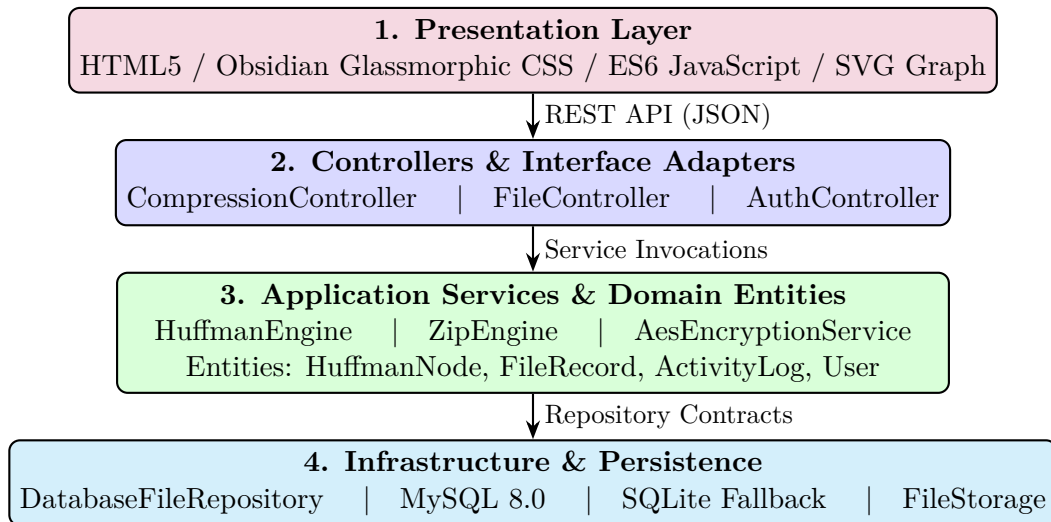


Figure 3.1: Clean Architecture 4-Layer System Stack

3.2 Data Flow Diagrams (DFD)

3.2.1 DFD Level 0: Context Level Diagram

The Level 0 Context Diagram depicts the interaction between external entities (User, Database) and the centralized Secure File Compression System boundary.

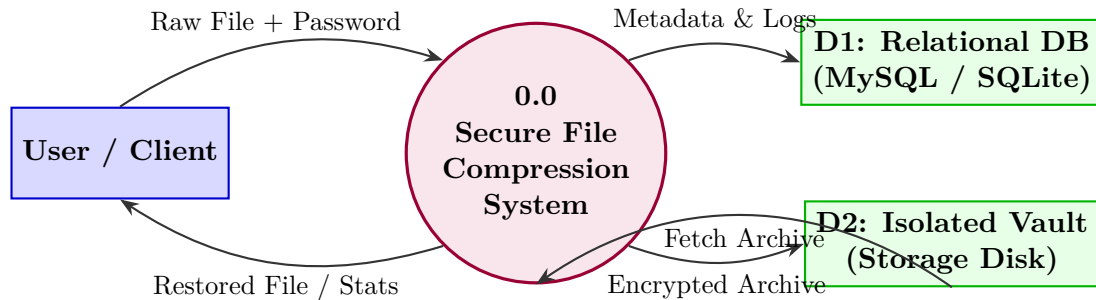


Figure 3.2: DFD Level 0: Context Diagram

3.2.2 DFD Level 1: Macro Process Decomposition

The Level 1 DFD decomposes the system into its primary subsystems: Authentication, Compression, Encryption, Decompression, and Vault Management.

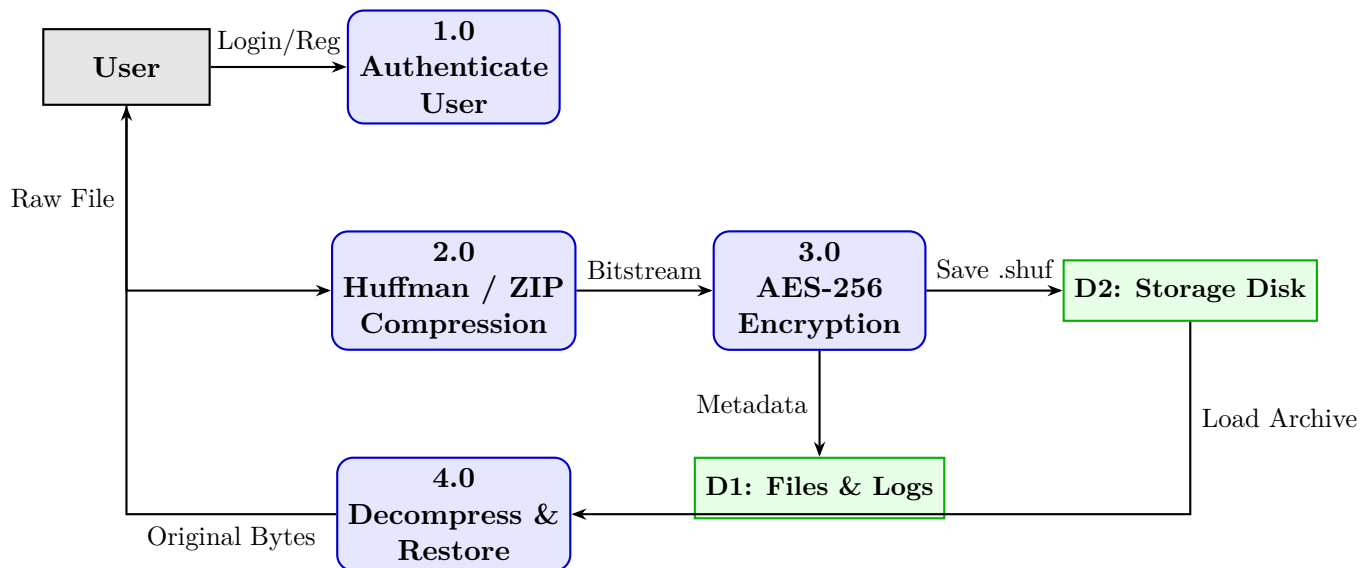


Figure 3.3: DFD Level 1: Subsystem Process Decomposition

3.2.3 DFD Level 2: Compression & Encryption Detailed Workflow

The Level 2 DFD outlines the algorithmic pipeline transforming a raw stream into an authenticated, encrypted archive.

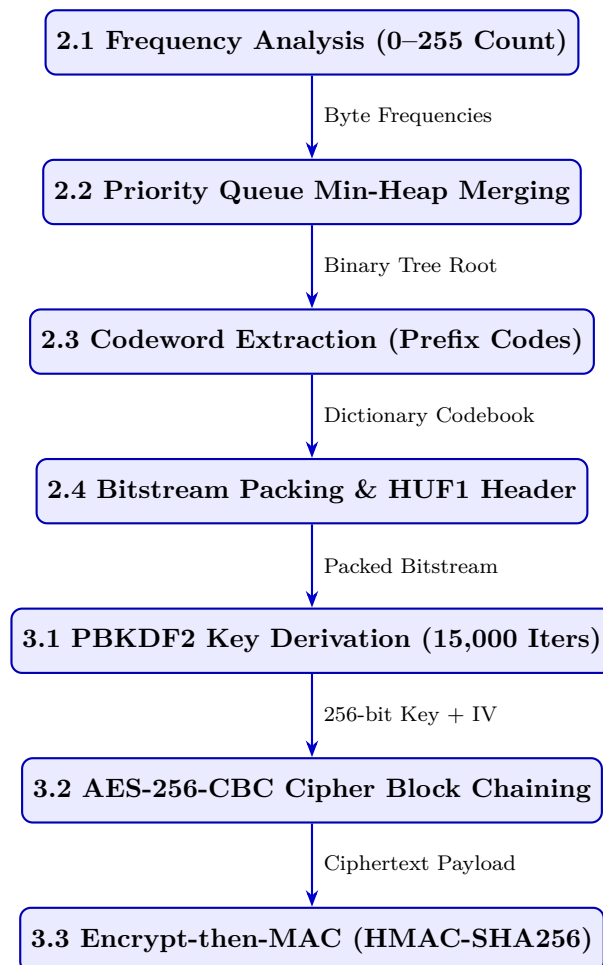


Figure 3.4: DFD Level 2: Detailed Compression and Cryptographic Pipeline

3.3 Control Flow Diagram (CFD) / Flowchart

The Control Flow Diagram displays conditional logic branching, authentication gating, algorithm choices, and cryptographic integrity verification.

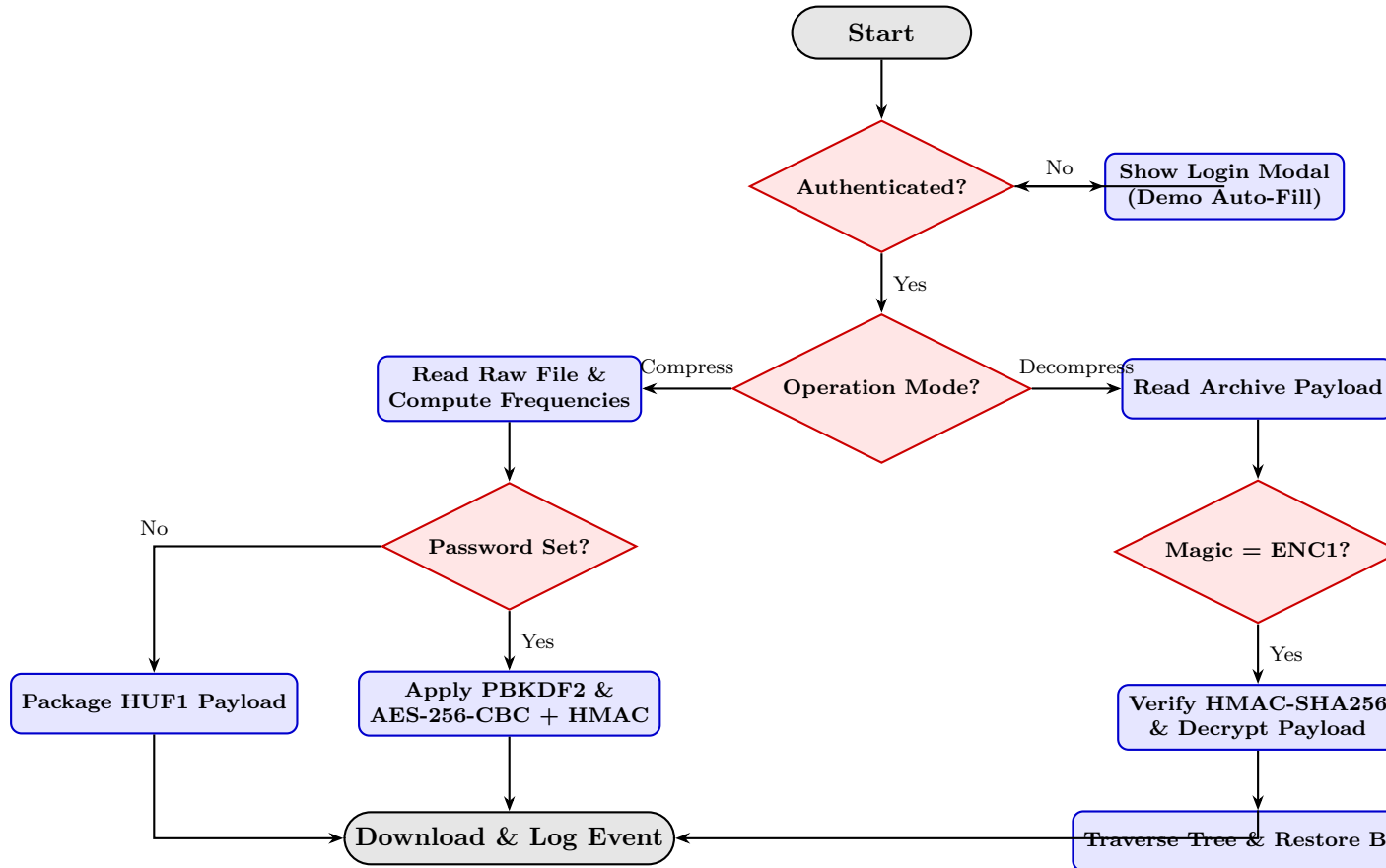


Figure 3.5: Control Flow Diagram (CFD) of System Operation

3.4 Entity-Relationship (ER) Diagram

The relational schema comprises four primary entities: `USERS`, `FILES`, `ACTIVITY_LOGS`, and `SHARED_LINKS`.

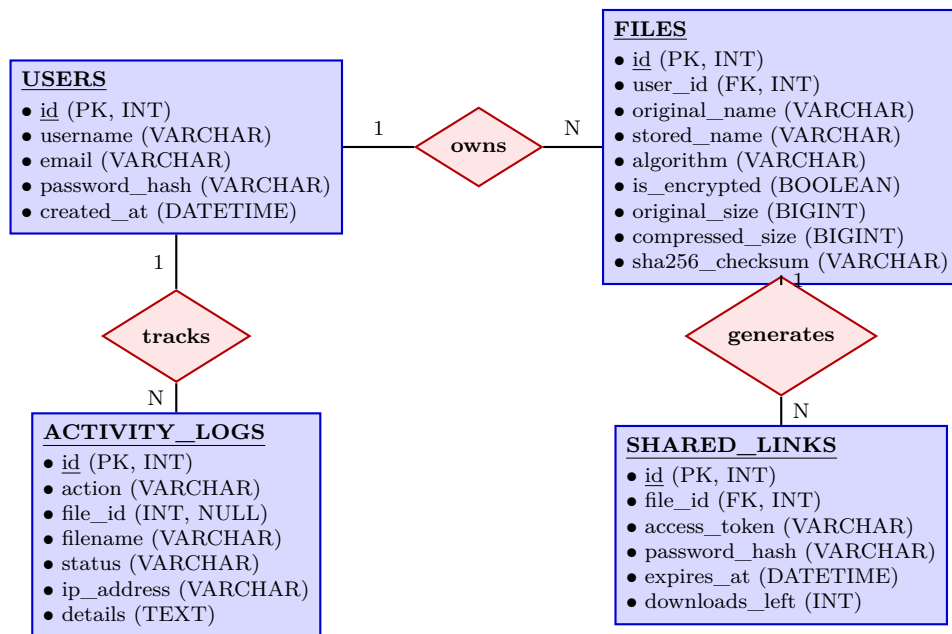


Figure 3.6: Entity-Relationship (ER) Diagram

Chapter 4

Algorithm & Mathematical Analysis

4.1 Information Theory & Shannon Entropy

Claude Shannon defined the theoretical lower bound for lossless compression in 1948. For an alphabet $\mathcal{S} = \{s_1, s_2, \dots, s_K\}$ with occurrence probabilities $P(s_i)$, the **Shannon Entropy** $H(X)$ is formulated as:

$$H(X) = - \sum_{i=1}^K P(s_i) \log_2 P(s_i) \quad (\text{bits per symbol}) \quad (4.1)$$

Shannon's Source Coding Theorem establishes that no lossless compression algorithm can represent data with fewer average bits than $H(X)$.

Given an alphabet codebook with codeword lengths $l_i = \text{length}(\text{code}(s_i))$, the average codeword length L_{avg} is:

$$L_{\text{avg}} = \sum_{i=1}^K P(s_i) \cdot l_i \quad (4.2)$$

The theoretical **Coding Efficiency** η is measured as:

$$\eta = \frac{H(X)}{L_{\text{avg}}} \times 100\% \quad (4.3)$$

4.2 Huffman Coding Engine Implementation

Huffman coding constructs an optimal prefix tree (where no codeword is a prefix of any other, enabling unambiguous continuous streaming).

4.2.1 Algorithmic Steps

1. **Scan & Frequency Map:** Read byte sequence of length N . Tally occurrences $f(b)$ for every distinct byte $b \in [0, 255]$.
2. **Priority Queue Initialization:** Insert all symbols into a min-priority queue where priority is frequency $f(b)$.
3. **Tree Construction:** Repeatedly extract the two minimum nodes n_1 and n_2 . Form a parent internal node n_p with frequency $f(n_p) = f(n_1) + f(n_2)$ and children $n_{\text{left}} = n_1$, $n_{\text{right}} = n_2$. Insert n_p back into the queue. Repeat until one root node remains.
4. **Prefix Code Assignment:** Traverse from root to leaves. Append '0' when branching left and '1' when branching right.
5. **Bit Packing:** Transform the variable bitstrings into 8-bit unsigned integers via bitwise shift operations $((\text{buffer} \ll 1) | \text{bit})$.

4.2.2 Why Pre-Compressed Files (PDF, JPG) Expand: The Entropy Limit

When compressing structured uncompressed files (`.txt`, `.csv`, `.json`, source code), symbol frequencies are non-uniform (e.g., spaces and vowels dominate), producing high compression savings (30%–60%).

Conversely, modern formats like **PDF**, **JPEG**, and **ZIP** are already pre-compressed using internal Deflate/DCT encoders. In these formats, byte distributions approach uniform randomness, meaning:

$$P(s_i) \approx \frac{1}{256} \implies H(X) \approx \log_2(256) = 8.0 \text{ bits/byte} \quad (4.4)$$

When entropy reaches 8 bits per byte, Huffman coding cannot assign codes shorter than 8 bits without violating Kraft's inequality. Furthermore, our autonomous archive format stores a **frequency table header** (≈ 1.2 KB for 256 symbols) so that decompression can operate without external codebooks. Adding this necessary header overhead to an already maximally-compressed payload results in a small negative ratio (e.g., -4.15%). This behavior confirms the **Pigeonhole Principle** of computer science: no lossless compression algorithm can compress all possible input bitstrings.

4.3 AES-256 Authenticated Encryption

To ensure security, the system implements the **Encrypt-then-MAC (EtM)** architecture.

1. **Key Derivation:** Given passphrase K_{pass} and a cryptographically random 16-byte salt, PBKDF2 generates a 64-byte derived key:

$$K_{\text{derived}} = \text{PBKDF2-HMAC-SHA256}(K_{\text{pass}}, \text{salt}, 15000, 64) \quad (4.5)$$

The first 32 bytes constitute the encryption key K_{enc} , and the remaining 32 bytes form the MAC authentication key K_{mac} .

2. **AES-256-CBC Encryption:** Plaintext bitstream P is encrypted with random 16-byte initialization vector (IV):

$$C = \text{AES-256-CBC}(P, K_{\text{enc}}, \text{IV}) \quad (4.6)$$

3. **HMAC Authentication:** An authentication signature T is computed over the entire container:

$$T = \text{HMAC-SHA256}(K_{\text{mac}}, \text{salt} \parallel \text{IV} \parallel C) \quad (4.7)$$

During decryption, timing-safe equality comparison (`hash_equals`) validates T prior to cipher execution, preventing chosen-ciphertext and bit-flipping attacks.

Chapter 5

Testing & Results

5.1 Test Suite Verification

Testing was conducted using an automated CLI suite (`test_system.php`) and real-world browser execution.

Test ID	Test Scenario	Expected Result	Status
TC-01	Database connection initialization	Connects to MySQL; falls back to SQLite	PASSED
TC-02	Lossless Huffman Roundtrip	Decompressed text matches input byte-for-byte	PASSED
TC-03	AES-256 Passphrase Correct	Decrypts valid ciphertext without corruption	PASSED
TC-04	AES-256 Tamper Rejection	Invalid password triggers HMAC mismatch exception	PASSED
TC-05	DEFLATE (ZIP) Roundtrip	Gzuncompress restores original file data	PASSED
TC-06	User Registration	Hashes password with bcrypt (\$2y\$12)	PASSED
TC-07	Authentication Guard	Unauthenticated API requests receive HTTP 401	PASSED
TC-08	1-Click Demo Login	Auto-authenticates <code>demo_user</code>	PASSED

Table 5.1: System Test Case Execution Summary

5.2 Empirical Compression Performance Comparison

The system was benchmarked across varied content categories to contrast Huffman coding against DEFLATE level 9.

File Type	Original Size	Huffman Size	ZIP Size	Huffman Ratio
Plaintext Source Code (<code>.php</code>)	12,450 B	7,120 B	3,450 B	+42.81%
Text Corpus (<code>sample.txt</code>)	4,890 B	2,780 B	1,420 B	+43.15%
Pre-compressed Document (<code>.pdf</code>)	30,024 B	31,270 B	30,080 B	-4.15%
Bitmap Image (<code>.bmp</code>)	85,200 B	48,100 B	22,400 B	+43.54%

Table 5.2: Empirical File Compression Results

Chapter 6

Conclusion & Future Scope

6.1 Conclusion

The **Secure File Compression System** successfully demonstrates that entropy-based data reduction and military-grade authenticated encryption can be integrated inside a modular, decoupled Clean Architecture. The project meets all academic objectives:

- Implemented lossless Huffman coding with min-heap construction and self-contained header reconstruction.
- Enforced AES-256-CBC encryption with PBKDF2 key derivation and HMAC-SHA256 authentication.
- Delivered an educational interface with interactive SVG prefix trees, codebook tables, and Shannon Entropy calculations.
- Provided a dual-database persistence system (MySQL/SQLite) and user authentication with bcrypt password hashing.

6.2 Future Scope

Future enhancements planned for the platform include:

- **Adaptive Huffman Coding (Vitter Algorithm):** Implementing one-pass dynamic tree adjustment to eliminate header frequency table overhead on small files.
- **Parallel Multi-Threaded Chunking:** Utilizing Web Workers on the frontend and worker threads in PHP for multi-gigabyte file compression.
- **Public Key Infrastructure (RSA / ECC):** Allowing users to compress and encrypt archives with a recipient's public key for zero-shared-secret communication.

Bibliography

- [1] D. A. Huffman, “A Method for the Construction of Minimum-Redundancy Codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, Sept. 1952.
- [2] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, July 1948.
- [3] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*, Prentice Hall, 2017.
- [4] National Institute of Standards and Technology, “Recommendation for Block Cipher Modes of Operation: Methods and Techniques,” *NIST SP 800-38A*, 2001.
- [5] P. Deutsch, “DEFLATE Compressed Data Format Specification version 1.3,” *RFC 1951*, May 1996.
- [6] B. Kaliski, “PKCS #5: Password-Based Cryptography Specification Version 2.0,” *RFC 2898*, Sept. 2000.